

Python + Ollama + TinyLlama

Step-by-step local AI setup for every student

Python 3.11+

Ollama 0.3+

TinyLlama 1.1B



Duration	~20 minutes total install time
Disk Space	~2 GB (TinyLlama model)
RAM Required	4 GB minimum 8 GB for Llama3
OS Support	Windows 10/11 • macOS 12+ • Ubuntu 20+

What's inside this guide

Section 1	Install Python 3.11+
Section 2	Install VS Code + Python Extension
Section 3	Install Ollama
Section 4	Pull the TinyLlama Model
Section 5	Create Your Project Folder
Section 6	Quick-Test Commands
Section 7	TinyLlama vs Llama3 — The Key Difference
Section 8	Install & Use phi3:mini (Microsoft's Compact Model)
Section 9	Troubleshooting Common Errors

1 Install Python 3.11+

Python is the programming language we will use to talk to the AI model. We need version **3.11 or newer**. Follow the steps for your operating system below.

Windows

1	Open your browser and go to python.org Navigate to https://www.python.org/downloads/ — it will auto-detect Windows.
2	Click the big yellow 'Download Python 3.x.x' button This downloads the official installer (.exe file) to your Downloads folder.
3	Run the installer — CRITICAL: tick 'Add Python to PATH' At the very bottom of the first screen you will see a checkbox. Tick it BEFORE clicking Install Now.
4	Click 'Install Now' and wait Installation takes about 2–3 minutes. Click 'Close' when it says Setup was successful.
5	Verify the install Open Command Prompt (Win + R → type cmd → Enter) and run: <code>python --version</code>

```
# Open Command Prompt and run:
python --version
# Expected output:
# Python 3.11.x (or newer)
```

■ **NOTE:** Must see 'Add Python to PATH' checkbox — without it, Python won't work from the terminal.

macOS

1	Install Homebrew (if not installed) Open Terminal (Cmd+Space → Terminal) and paste the one-liner from brew.sh
2	Install Python via Homebrew Run: <code>brew install python@3.11</code>
3	Verify the install Run: <code>python3 --version</code> — you should see Python 3.11.x

```
# macOS install commands:
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install python@3.11
python3 --version
```

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install python3.11 python3-pip -y
python3 --version
```

2 Install VS Code + Python Extension

VS Code is the code editor we use. It gives you syntax highlighting, auto-complete, and an integrated terminal all in one window.

1	Download VS Code Go to https://code.visualstudio.com/ and click the big blue Download button for your OS.
2	Run the installer Windows: double-click the .exe. macOS: drag to Applications. Linux: use the .deb/.rpm package.
3	Open VS Code and install the Python extension Click the Extensions icon (Ctrl+Shift+X), search 'Python', install the Microsoft one (first result).
4	Set your Python interpreter Press Ctrl+Shift+P → type 'Python: Select Interpreter' → choose Python 3.11+.
5	Open the integrated terminal Press Ctrl+` (backtick) to open the terminal inside VS Code — you'll use this for all commands.

✓ **TIP:** Always use VS Code's integrated terminal — it automatically uses your project folder as the working

3 Install Ollama

Ollama is the engine that runs AI models locally on your computer. Think of it as a local server: it downloads models and serves them through a simple API that your Python code can call — no internet needed after setup.

Windows

1	Download Ollama Go to https://ollama.com and click 'Download for Windows'.
2	Run OllamaSetup.exe Double-click the installer. It installs silently in about 30 seconds.
3	Look for the Ollama icon in the system tray Bottom-right corner of your screen (^). A llama icon means Ollama is running. Right-click to see options.
4	Restart your PC if the command isn't recognised After restarting, open a new terminal and run: <code>ollama --version</code>

```
ollama --version  
  
# Expected output:  
# ollama version 0.x.x
```

macOS

1	Download Go to https://ollama.com and click 'Download for Mac'.
2	Open the .dmg and drag Ollama to Applications Standard Mac install flow.
3	Launch Ollama from Applications A llama icon appears in your menu bar — Ollama is now running.
4	Verify Open Terminal and run: <code>ollama --version</code>

Linux

```
# One-line install:  
curl -fsSL https://ollama.com/install.sh | sh  
  
# Ollama starts as a background service automatically.  
# Verify:  
ollama --version
```

■ NOTE: Ollama must be RUNNING (system tray icon visible) before your Python code can use it.

4 Pull the TinyLlama Model

Pulling a model means downloading it from Ollama's model library to your local machine. TinyLlama is about **638 MB** — small enough to run on any laptop with 4 GB RAM. While it downloads, your teacher will explain what a local LLM is.

- 1 Open a terminal (Command Prompt / Terminal / VS Code terminal)**
Any terminal will work.
- 2 Run the pull command**
Type the command below and press Enter. Download takes 1–5 min depending on Wi-Fi.
- 3 Watch the progress bar**
Ollama shows a live download bar. Wait until you see 'success'.
- 4 Test TinyLlama interactively**
After pulling, you can chat with it directly in the terminal.

```
# Download TinyLlama (638 MB):
ollama pull tinyllama

# When done, test it in the terminal:
ollama run tinyllama

# You'll see a >>> prompt — type your message and press Enter:
>>> What is a large language model?

# Type /bye to exit the interactive session
/bye
```

TinyLlama vs Llama3 — Quick Reference

Feature	TinyLlama	Llama3 8B
Parameters	1.1 billion	8 billion
Download Size	638 MB	~4.7 GB
RAM Needed	4 GB	8 GB
Response Quality	Good for learning	Much more detailed
Response Speed	Very fast	Moderate
Python Code	Same	Same
Switch command	model="tinyllama"	model="llama3"

✓ **TIP:** The Python code is IDENTICAL — just swap "tinyllama" with "llama3" when you have 8 GB RAM.

5 Create Your Project Folder

Keeping all your AI project files in one folder is important. The **ollama Python library** must be installed inside this folder's environment.

Windows (Command Prompt)

```
# Navigate to your Desktop:
cd %USERPROFILE%\Desktop

# Create the project folder:
mkdir ai-agent

# Go into it:
cd ai-agent

# Install the Ollama Python library:
pip install ollama

# Open the folder in VS Code:
code .
```

macOS / Linux (Terminal)

```
# Navigate to Desktop:
cd ~/Desktop

# Create and enter the project folder:
mkdir ai-agent && cd ai-agent

# Install the Ollama Python library:
pip3 install ollama

# Open in VS Code:
code .
```

✖ **ERROR FIX:** If 'pip install ollama' gives an error, make sure you are **INSIDE** the ai-agent folder first.

✓ **TIP:** 'code .' opens VS Code in the current folder — run this every time you start working.

6 Quick-Test: First AI Response in Python

Create a file called **test.py** inside your ai-agent folder. Copy the code below, save it, and run it. You should see a response from TinyLlama.

test.py — your first AI program

```
import ollama

# Send a message to TinyLlama
response = ollama.chat(
    model='tinyllama',
    messages=[
        {'role': 'user', 'content': 'Say hello in 10 words or less'}
    ]
)

# Print only the text part of the response
print(response['message']['content'])
```

Run it from the VS Code terminal:

```
# Windows:
python test.py

# macOS / Linux:
python3 test.py

# Expected output (something like):
# Hello! How can I help you today?
```

Code explained — line by line

Line	What it does
<code>import ollama</code>	Loads the ollama library you installed with pip.
<code>ollama.chat(...)</code>	Calls the chat function — like sending a message to the AI.
<code>model='tinyllama'</code>	Tells Ollama which model to use. Change to 'llama3' for the bigger model.
<code>messages=[...]</code>	A list of messages. Each has a 'role' (user/assistant) and 'content' (the text).
<code>response['message']['content']</code>	Extracts just the text reply from the full response object.

7 TinyLlama vs Llama3 — The Full Picture

Understanding why we use TinyLlama in class will help you choose the right model for your own projects.

- **Parameters are 'knowledge weights'**

TinyLlama has 1.1 billion parameters; Llama3 8B has 8 billion. More parameters = more nuanced understanding and longer, more accurate answers.

- **The Python code is 100% identical**

Ollama abstracts away all complexity. To switch models, change ONE word in your code: 'tinyllama' → 'llama3'.

- **TinyLlama is perfect for learning**

Faster responses, runs on any laptop, and teaches you the same programming patterns. The skills transfer directly.

- **Llama3 for production / real projects**

When you need higher quality answers, richer reasoning, or better instruction-following, upgrade to Llama3 on an 8 GB+ machine.

- **Both run 100% offline**

After the one-time download, neither model needs internet. Your data never leaves your computer.

```
# To use Llama3 instead — change only ONE line:

# BEFORE (TinyLlama):
response = ollama.chat(model='tinyllama', messages=[...])

# AFTER (Llama3):
response = ollama.chat(model='llama3', messages=[...])

# Pull Llama3 first (one time only, needs ~4.7 GB):
ollama pull llama3
```

8 Install & Use phi3:mini (Microsoft's Compact Model)

phi3:mini is a 3.8-billion-parameter model built by Microsoft. It punches well above its weight — delivering near-GPT-3.5-level reasoning in a package that only needs about **2.3 GB of disk space** and **4-6 GB RAM**. It sits perfectly between TinyLlama (speed) and Llama3 (quality).

Why use phi3:mini?

■ Instruction-tuned out of the box

phi3:mini follows complex multi-step instructions much more reliably than TinyLlama — great for agents and structured outputs.

■ Better reasoning for its size

Microsoft trained it on high-quality textbook-style data, so it reasons through problems step-by-step even at 3.8B params.

■ Runs on 4 GB RAM laptops

Unlike Llama3 (needs 8 GB), phi3:mini fits comfortably on most student laptops while still giving quality answers.

■ Same Python code — one word change

Replace 'tinyllama' with 'phi3:mini' and everything else stays identical.

Step 1 - Pull phi3:mini (one-time download ~2.3 GB)

```
# Download phi3:mini:
ollama pull phi3:mini

# Check it downloaded successfully:
ollama list

# You should see phi3:mini in the list
```

Step 2 - Test it interactively in the terminal

```
# Chat with phi3:mini directly:
ollama run phi3:mini

# Try a reasoning question:
>>> Explain what a Python list is in 2 sentences.

# Exit when done:
/bye
```

Step 3 - Use phi3:mini in Python

```
import ollama

# Only ONE word changes from the TinyLlama version:
response = ollama.chat(
    model='phi3:mini',          # <-- changed from 'tinylama'
    messages=[
        {'role': 'user', 'content': 'Explain Python functions in simple terms'}
    ]
)

print(response['message']['content'])
```

Three-Model Comparison - Which to use when?

Feature	TinyLlama	phi3:mini	Llama3 8B
Parameters	1.1B	3.8B	8B
Download Size	638 MB	~2.3 GB	~4.7 GB
RAM Needed	4 GB	4-6 GB	8 GB
Response Speed	Very fast	Fast	Moderate
Reasoning	Basic	Strong	Best
Instructions	Simple only	Multi-step	Complex
Best for	First programs	Agents/logic	Production
Pull command	tinylama	phi3:mini	llama3

✓ TIP: Start with TinyLlama to learn, move to phi3:mini for agents, use Llama3 when quality matters most.

9 Troubleshooting Common Errors

These are the five most common errors students encounter and their exact fixes.

Error: python not recognized

Symptom: python --version gives an error

Fix: Reinstall Python from python.org. On the FIRST screen of the installer, tick the checkbox labelled 'Add Python 3.x to PATH'. Then click Install Now.

```
python --version
# If still not working:
py --version      # Windows alternative
```

Error: ollama not recognized

Symptom: ollama --version gives an error

Fix: Ollama was installed but the PATH was not refreshed. Restart your PC completely (not just the terminal), then open a fresh terminal.

```
# After restarting:
ollama --version
```

Error: model not found / 404

Symptom: Running your Python script gives a model not found error

Fix: The model was not downloaded yet, or the name is misspelled. Re-run the pull command.

```
ollama pull tinyllama
# Check all downloaded models:
ollama list
```

Error: Connection refused

Symptom: Python script gives 'Connection refused' or 'Failed to connect'

Fix: Ollama is not running. Look for the llama icon in the Windows system tray (bottom-right). If missing, open Ollama from the Start Menu.

```
# Test if Ollama API is reachable:
curl http://localhost:11434/api/tags
# Should return a JSON list of models
```

Error: ModuleNotFoundError: No module named 'ollama'

Symptom: import ollama fails

Fix: The ollama Python package is not installed in the current environment. Make sure you are inside your ai-agent folder, then run pip install ollama.

```
cd Desktop\ai-agent      # Windows
cd ~/Desktop/ai-agent    # macOS/Linux
pip install ollama
python test.py
```

Quick-Reference Cheat Sheet

Task	Command
Check Python version	python --version
Check Ollama version	ollama --version
Download TinyLlama	ollama pull tinyllama
Download Llama3	ollama pull llama3
Chat in terminal	ollama run tinyllama
List downloaded models	ollama list
Install Python package	pip install ollama
Run a Python file	python test.py
Open folder in VS Code	code .

You're all set! Once Python, Ollama, and TinyLlama are installed, every AI program in this course will run on your local machine — no cloud, no subscription, no internet required.